

0.85. Plusieurs ennemis : un tableau de struct

Trois B, rangés dans `Position ennemis[3]`, partis d'un endroit au hasard ; une boucle `for` les fait tous avancer vers le A.

1. Ce qu'il faut comprendre

Trois poursuivants au lieu d'un. Plutôt que `bx`, `by`, `bx2`, `by2`, `bx3`, `by3`, on les range dans **un tableau de struct** : `Position ennemis[3]`; (le type `Position` du 0.80).

Ce qui est nouveau ici : `ennemis[i].x`. D'abord la **case** du tableau (`[i]`, l'ennemi n° i), **puis** la partie (`.x`, sa colonne). `ennemis[0].x` est la colonne du premier, `ennemis[2].y` la ligne du troisième.

Une boucle `for` passe sur les trois : le même code fait avancer chacun vers le A (le 0.79). Pour quatre ennemis, on changerait seulement le 3 en 4.

Au départ, au hasard (le 0.81.2) : chaque ennemi prend une colonne `hasard() % 20` sur les lignes du haut. Chaque partie commence différemment.

Ils finissent par se rejoindre sur le A : ils visent tous la même case.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1788 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      88 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
deplace_croix()  217 o ← main() — une fonction fournie par le langage
un_pas()        117 o ← main() — une fonction fournie par le langage
main()          348 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o ← image() — attend l'image suivante
AttendreVBlank  16 o ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases     12 o ← effacer() — efface des cases
Reste           10 o ← main() — le reste d'une division
```

```

CopierTuiles      18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
Hasard            25 o ← hasard() – tire un nombre au hasard
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            704 o ← CopierTuiles – la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser()        x 3    61 o sur place, dans deplace_croix(), un_pas(), main() + AdresseCase (10 o)
    effacer()      x 2    30 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o), EffacerCases (12 o)
    bouton()       x 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
    hasard()       x 2    6 o sur place, dans main() + Hasard (25 o)
    images()       x 2    4 o sur place, dans chaque_minuteur(), deplace_croix()
    image()        x 1    3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    un_pas()       x 1    41 o sur place, dans main()
    deplace_croix() x 1    19 o sur place, dans main()
    chaque_minuteur() x 1  10 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- LE TYPE Position (le 0.80) : une colonne et une ligne ----

#include <hasard>           // tire un nombre au hasard
#include <poser>            // pose une tuile sur une case du fond
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <chaque>           // répond 1 toutes les n millisecondes
#include <un_pas>           // fait faire un seul pas à une tuile
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <reste>            // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UN ENREGISTREMENT — POSITION

```

struct Position { uint8_t x, y; };

// ---- UN TABLEAU DE Position : trois ennemis ----
//
// Position ennemis[3];
// |      |      |
// |      |      +-- [3] : trois cases, numérotées 0, 1 et 2
// |      +----- le nom du tableau
// +----- chaque case est une Position (un x et un y)
//
// ennemis[0].x ennemis[0].y ← le 1er ennemi
// ennemis[1].x ennemis[1].y ← le 2e
// ennemis[2].x ennemis[2].y ← le 3e

```

3. UN TABLEAU — ENNEMIS

```
Position ennemis[3];
```

4. UNE VARIABLE — X

```
uint8_t x = 10;      // le A : en bas, (10, 12)
```

5. UNE VARIABLE — Y

```
uint8_t y = 12;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;      // la direction du prochain pas d'un ennemi
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE DÉPART : chaque ennemi au hasard, en haut de l'écran ----
    // i vaut 0, puis 1, puis 2 : la boucle s'occupe de chaque ennemi à son tour.
    for (uint8_t i = 0; i < 3; i++) {
        ennemis[i].x = hasard() % 20;          // sa colonne : de 0 à 19 (le 0.81.2)
        ennemis[i].y = hasard() % 3;           // sa ligne : de 0 à 2, tout en haut
        poser(ennemis[i].x, ennemis[i].y, ALPHABET[1]); // on l'y pose : un B
    }

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();

        if (chaque(700)) {                    // tous les 700 ms...
            for (uint8_t i = 0; i < 3; i++) { // ...chaque ennemi fait un pas vers le A
                // ennemis[i].x se lit en deux temps :
                //   ennemis[i] : d'abord la CASE du tableau, l'ennemi n° i
                //   .x           : puis la PARTIE, sa colonne
                //
                // La direction, comme au 0.79, mais avec les parties de ennemis[i] :
                sx = 0;
                if (x > ennemis[i].x) { sx = 1; }
                if (x < ennemis[i].x) { sx = -1; }
                sy = 0;
                if (y > ennemis[i].y) { sy = 1; }
                if (y < ennemis[i].y) { sy = -1; }
                if (sx != 0 || sy != 0) {
                    // un_pas range la nouvelle place dans ennemis[i].x et ennemis[i].y :
                    // la console sait ranger dans une partie de struct (le 0.80).
                    un_pas(ennemis[i].x, ennemis[i].y, ALPHABET[1], sx, sy);
                }
            }
        }

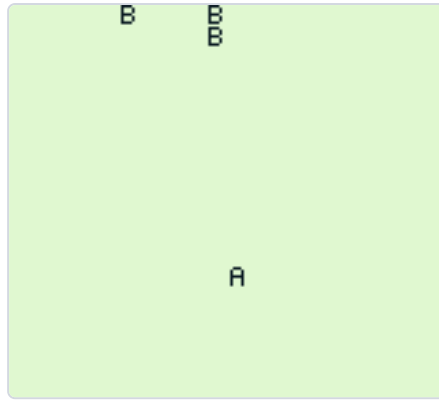
        deplace_croix(x, y, ALPHABET[0], 250); // le A, dessiné en dernier : par-dessus
    }
}
```

3. Ce qui se passe, étape par étape

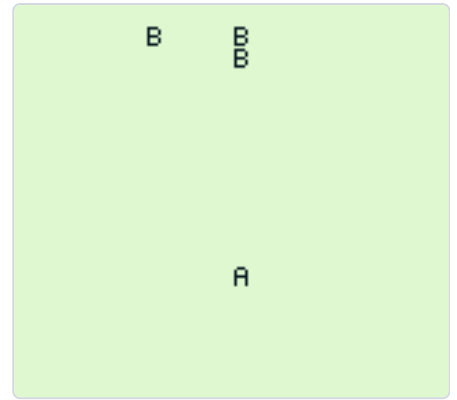
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



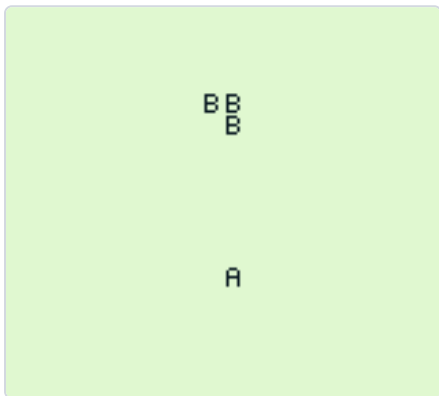
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Trois B partis d'en haut, à des places différentes à chaque partie, qui descendent tous vers le A.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ des B à l'écran au départ (3)
- ✓ ils sont descendus vers le A (ligne 8 ou plus bas)

Ensuite : **0.85.1. ALPHABET_GRAS — les lettres en gras** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++